

AI-103 Study Guide: Module 1.3 Enhanced

Develop a Generative AI Chat App with Microsoft Foundry

This is an ENHANCED version with critical API details filled in from TXT lecture

1. Exam Domain Mapping

AI-103 Exam Domain	Relevance	Why
Implement generative AI and agentic solutions	High	Covers building a chat app, selecting endpoint/SDK/API, generating responses, managing conversation context, and streaming
Plan and manage an Azure AI solution	Medium-High	Covers endpoint choice, SDK choice, authentication, project access, and production authentication considerations
Implement text analysis solutions	Low	Uses text prompts/responses, but focus is generative chat rather than specialist text analysis
Implement information extraction solutions	Low	Not the focus, though chat apps may later integrate RAG or tools
Implement computer vision solutions	Low	Not covered in this module

Exam focus: How client applications connect to Microsoft Foundry models and how to choose between endpoints, SDKs, APIs, and authentication methods.

2. What This Module Teaches

This module teaches how to build a generative AI chat application that connects to a model deployed in a Microsoft Foundry project.

Key message:

Building a chat app is not only about sending a prompt to a model. You must choose the correct endpoint, SDK, authentication approach, API style, conversation-state strategy, and response-handling pattern.

The module starts in the Foundry playground, then moves into code using the OpenAI SDK, Responses API, ChatCompletions API, conversation tracking, streaming, and asynchronous clients.

3. Key Concepts

3.1 Endpoints (CRITICAL DISTINCTION)

Each Microsoft Foundry project exposes **two different endpoints** for model consumption.

Endpoint	Format	Purpose	Use When
Project endpoint	<code>https://{resource-name}.services.ai.azure.com/api/projects/{project-name}</code>	Access Foundry-native capabilities: agents, tracing, datasets, indexes, connections, project configuration	You need Foundry-specific features like agents, evaluations, tracing, or Foundry direct models
Azure OpenAI endpoint	<code>https://{resource-name}.openai.azure.com/openai/v1</code>	OpenAI-compatible model inference	You need broad OpenAI API compatibility for model inference with OpenAI SDK patterns

CRITICAL EXAM POINT: These are NOT interchangeable. Choose based on what features you need.

Exam memory: For broad OpenAI API support with Foundry models, use the **Azure OpenAI endpoint**.

3.2 SDK Choices

SDK	Python Package	Best For	Key Difference
Microsoft Foundry SDK / Azure AI Projects	<code>azure-ai-projects</code>	Project-level features: agents, evaluations, tracing, datasets, indexes, connections, governance	Foundry-native operations
OpenAI SDK	<code>openai</code>	Model inference, Responses API, ChatCompletions API, image generation	Maximum OpenAI API compatibility

When to use Foundry SDK:

- * Building Foundry agents
- * Need evaluations or tracing
- * Need project configuration access

* Need Foundry direct models

When to use OpenAI SDK:

- * Simple model inference
- * Existing OpenAI code to adapt
- * Maximum portability
- * ChatCompletions or Responses API

Exam memory: If the question asks which Python package is used for the Microsoft Foundry SDK, choose **azure-ai-projects**.

3.3 Authentication (PRODUCTION CRITICAL)

Azure Credential Types (7 Credential Classes)

Credential Class	Best For	When to Use	In DefaultAzureCredential Chain?
AzureKeyCredential	Development/Testing	Quick local testing; never production	■ NO
DefaultAzureCredential	Local dev to production	Recommended — works everywhere	N/A (it IS the chain)
AzureCliCredential	Local development	Local dev with Azure CLI installed	■ YES (step 4)
EnvironmentCredential	CI/CD pipelines	Automated deployments with env vars	■ YES (step 1)
ManagedIdentityCredential	Azure resources (VMs, App Service)	Running in Azure infrastructure	■ YES (step 3)
InteractiveBrowserCredential	User login	User-facing applications	■ YES (step 6)
DeviceCodeCredential	Limited devices	IoT/limited-input devices	■ NO

Important: Only 4 of the 7 credential classes are part of the DefaultAzureCredential fallback chain.

DefaultAzureCredential Fallback Chain (EXAM CRITICAL)

DefaultAzureCredential tries 6 authentication methods in this order until one succeeds:

Step 1: Environment variables (AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID)
→ Uses EnvironmentCredential

↓

Step 2: Workload Identity (internal method, not a separate credential class)

↓

Step 3: Managed Identity (for Azure-hosted resources)
→ Uses `ManagedIdentityCredential`

↓

Step 4: Azure CLI (`az login`)
→ Uses `AzureCliCredential`

↓

Step 5: Azure PowerShell (internal method, not a separate credential class)

↓

Step 6: Interactive browser login (opens browser window)
→ Uses `InteractiveBrowserCredential`

Mapping: 4 credential classes in the chain (`EnvironmentCredential`, `ManagedIdentityCredential`, `AzureCliCredential`, `InteractiveBrowserCredential`) + 2 internal methods (`Workload Identity`, `Azure PowerShell`)

Exam point: `Workload Identity` and `Azure PowerShell` are methods `DefaultAzureCredential` tries internally, but they are NOT separate credential classes you instantiate directly.

Detailed Authentication Methods (How, Where, Entra ID Relationship)

1. `DefaultAzureCredential` (RECOMMENDED)

How it looks:

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

credential = DefaultAzureCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Where it fits:

- * Works everywhere: local dev, CI/CD pipelines, Azure resources
- * Automatically detects the environment and uses the appropriate authentication method
- * Falls back through the chain until one succeeds

Entra ID relationship:

- * ■ Uses Entra ID (tries multiple Entra ID methods in the fallback chain)
- * Connects to Entra ID via: env vars → Managed Identity → Azure CLI → browser login
- * No explicit Entra ID configuration needed — it handles it automatically

Setup requirements: None for local dev (just `az login` first). For CI/CD, set environment variables.

Exam point: Best practice because it's environment-agnostic.

2. `AzureKeyCredential` (Simple but NOT for production)

How it looks:

```
from azure.core.credentials import AzureKeyCredential
from openai import AzureOpenAI

credential = AzureKeyCredential(api_key="your-api-key-here")

client = AzureOpenAI(
    endpoint=azure_openai_endpoint,
    api_key="your-api-key-here"
)
```

Where it fits:

- * Quick local testing and prototyping
- * Not recommended for production (secrets hardcoded or exposed)
- * Sometimes used in development scripts

Entra ID relationship:

- * ■ Does NOT use Entra ID
- * Uses raw API key instead
- * Bypasses all Entra ID authentication

Setup requirements: Just get the API key from Azure portal.

Exam point: Simple but insecure — never use in production.

3. **EnvironmentCredential** (CI/CD pipelines)

How it looks:

```
from azure.identity import EnvironmentCredential
import os

# Set BEFORE running (via pipeline secrets or env vars):
# AZURE_CLIENT_ID
# AZURE_CLIENT_SECRET
# AZURE_TENANT_ID

credential = EnvironmentCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Environment variables required:

```
export AZURE_CLIENT_ID="<app-id>"
export AZURE_CLIENT_SECRET="<client-secret>"
export AZURE_TENANT_ID="<tenant-id>"
```

Where it fits:

- * CI/CD pipelines (GitHub Actions, Azure Pipelines, Jenkins)

- * Automated deployments
- * Serverless environments with env var support
- * Secrets stored securely in pipeline variables, not in code

Entra ID relationship:

- * ■ Uses Entra ID
- * Authenticates as a **service principal** registered in Entra ID
- * Environment variables contain service principal credentials that Entra ID validates
- * Service principal is created in Azure AD/Entra ID with specific permissions

Setup requirements:

1. Create a service principal in Entra ID
2. Store its credentials (client ID, secret, tenant ID) as pipeline secrets
3. Set environment variables before running code

Exam point: Standard for automation — service principal credentials in secure pipeline variables.

4. AzureCliCredential (Local development)

How it looks:

```
from azure.identity import AzureCliCredential

credential = AzureCliCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Where it fits:

- * Local development (your laptop/machine)
- * After you've run **az login** once
- * Convenient for developers

Prerequisites:

```
# Install Azure CLI (if not already installed)
az login
# Browser opens → sign in with your account → credentials cached
```

Entra ID relationship:

- * ■ Uses Entra ID
- * Reads cached credentials from Azure CLI login
- * Azure CLI stores your Entra ID tokens locally (in `~/.azure/`)
- * When you run **az login**, you authenticate against Entra ID
- * Your user account (not a service principal) is authenticated

Setup requirements:

1. Install Azure CLI
2. Run **az login** once (opens browser)

3. Credentials automatically cached and reused

Exam point: Convenient for local dev — uses your personal Azure account credentials cached by CLI.

5. **ManagedIdentityCredential** (Azure resources only)

How it looks:

```
from azure.identity import ManagedIdentityCredential

credential = ManagedIdentityCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Where it fits:

- * Running inside Azure (VMs, App Service, Functions, ACI, AKS)
- * NO code changes needed — just runs automatically when in Azure
- * Cannot be used locally (will fail if not in Azure)

How it works:

- * Azure infrastructure automatically provides credentials
- * No secrets to manage or store
- * Identity managed by Azure

Entra ID relationship:

- * ■ Uses Entra ID (but you don't see it)
- * Azure automatically registers the resource (VM, App Service, etc.) in Entra ID as a **managed identity**
- * When code runs in Azure, Azure provides Entra ID tokens automatically
- * You don't explicitly authenticate — Azure does it behind the scenes

Setup requirements:

1. Enable Managed Identity on the Azure resource (VM, App Service, etc.)
2. Grant the managed identity permissions to Foundry
3. Code automatically works — no credentials needed

Exam point: Best for Azure-hosted resources — zero secrets, automatic Entra ID integration.

6. **InteractiveBrowserCredential** (User login)

How it looks:

```
from azure.identity import InteractiveBrowserCredential

credential = InteractiveBrowserCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Where it fits:

- * User-facing applications (web apps, desktop apps)
- * When a real person needs to log in
- * Opens a browser window for user authentication

How it works:

1. Code triggers login
2. Browser opens automatically
3. User enters credentials (or uses existing Entra ID session)
4. Credentials returned to app, browser closes
5. App uses credentials to authenticate

Entra ID relationship:

- * ■ Uses Entra ID
- * Redirects user to Entra ID login page
- * User authenticates directly against Entra ID
- * Entra ID returns token to app
- * User account (not service principal) is authenticated

Setup requirements:

1. Register application in Entra ID
2. Configure redirect URI (typically <http://localhost:port>)
3. Code creates InteractiveBrowserCredential

Exam point: For user-facing apps — delegates authentication to Entra ID's browser login flow.

7. DeviceCodeCredential (Limited-input devices)**How it looks:**

```
from azure.identity import DeviceCodeCredential

credential = DeviceCodeCredential()

client = AIProjectClient(
    endpoint=project_endpoint,
    credential=credential
)
```

Output when running:

To sign in, use a web browser to open the page <https://microsoft.com/devicelogin> and enter the code ABC123DEF to authenticate.

Where it fits:

- * IoT devices, smart TVs, kiosks (no browser)
- * Limited-input scenarios (can't open browser)
- * User authenticates on a DIFFERENT device

How it works:

1. App displays device code and URL
2. User goes to browser on phone/laptop
3. User enters device code
4. App receives authentication token
5. Authentication complete

Entra ID relationship:

- * ■ Uses Entra ID
- * Device code flow is an Entra ID authentication protocol
- * User authenticates against Entra ID via browser on different device
- * Device receives token after user completes authentication

Setup requirements:

1. Register app in Entra ID
2. Enable "public client" flows
3. Code creates DeviceCodeCredential

Exam point: For devices without browser — uses Entra ID device code flow.

Entra ID Summary Across All Methods

Method	Uses Entra ID?	Type	Identity
DefaultAzureCredential	■ YES	Automatic	Varies (env vars → MI → CLI → browser)
AzureKeyCredential	■ NO	Raw key	N/A (no identity)
EnvironmentCredential	■ YES	Service Principal	Service principal in Entra ID
AzureCliCredential	■ YES	User account	Your personal Azure account
ManagedIdentityCredential	■ YES	Managed Identity	Azure resource's managed identity in Entra ID
InteractiveBrowserCredential	■ YES	User account	User logging in via browser
DeviceCodeCredential	■ YES	User account	User on different device

Key insight: 6 of 7 methods use Entra ID. Only AzureKeyCredential bypasses it entirely.

Production Authentication Rules

Rule	Why
■ Use DefaultAzureCredential in production	Seamless Managed Identity support in Azure
■ Use Entra ID (via Managed Identity) in Azure	No secrets to manage or expose
■ Use EnvironmentCredential for CI/CD	Secrets in secure pipeline variables

Rule	Why
■ Store API keys in Azure Key Vault if using AzureKeyCredential	Never hard-code secrets
■ Avoid hard-coded API keys in production	Security risk; use credential objects instead
■ Don't use AzureKeyCredential in production	Credentials embedded in code

Entra ID Scope URL for Foundry

When using any credential with Foundry, include the scope:

```
from azure.identity import DefaultAzureCredential

credential = DefaultAzureCredential()

# Foundry uses this scope URL
scope = "https://ai.azure.com/.default"
```

3.4 Responses API (RECOMMENDED APPROACH)

The **Responses API** is the recommended approach for most new Microsoft Foundry chat development.

Key Features:

Feature	Meaning
Stateful conversations	Can maintain context using <code>previous_response_id</code>
Unified experience	Combines chat completions and Assistants API patterns
Foundry direct model support	Works with models hosted directly in Microsoft Foundry
Simple syntax	Uses <code>instructions</code> and <code>input</code> parameters
Streaming support	Allows incremental response display
Async support	Supports non-blocking application behavior

Core method: `client.responses.create()`

Response Properties:

```
response = client.responses.create(
    model="gpt-4.1",
    instructions="You are a helpful AI assistant.",
    input="What is Microsoft Foundry?"
)

# Response object properties:
```

```
response.output_text      # Generated text response
response.id               # Unique identifier for this response
response.status           # Response status (e.g., "completed")
response.usage.total_tokens # Token usage information
response.model            # Model used to generate the response
```

Exam memory: If the question asks which method generates responses with the Responses API, choose `client.responses.create()`.

3.5 ChatCompletions API (LEGACY - STILL TESTED)

The **ChatCompletions API** is the older, widely adopted API pattern.

Key Features:

Feature	Meaning
Message array	Uses JSON messages with roles: system , user , and assistant
Manual context tracking	You must send prior messages each turn to retain context
Cross-platform familiarity	Common across many models and platforms
Useful for legacy code	Often needed for maintenance and compatibility

Core method: `client.chat.completions.create()`

Message Structure:

```
completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "When was Microsoft founded?"}
    ]
)

# Access response
response_text = completion.choices[0].message.content
```

Exam memory: Responses API is recommended for new projects, but ChatCompletions remains useful for existing code and OpenAI-compatible patterns.

3.6 Conversation Context Management

Responses API Pattern (Using `previous_response_id`)

```
last_response_id = None

# Turn 1
response1 = client.responses.create(
```

```
    model=model_deployment,
    instructions="You are a helpful assistant.",
    input="What is machine learning?",
    previous_response_id=last_response_id
)
last_response_id = response1.id

# Turn 2 - context automatically included via response ID
response2 = client.responses.create(
    model=model_deployment,
    instructions="You are a helpful assistant.",
    input="Can you give me an example?",
    previous_response_id=last_response_id # Links to previous turn
)
last_response_id = response2.id
```

Advantage: Simpler syntax, automatic context tracking

Disadvantage: Token usage still increases with context

ChatCompletions Pattern (Manual Message Array)

```
conversation_messages = [
    {"role": "system", "content": "You are a helpful assistant."}
]

# Turn 1
conversation_messages.append(
    {"role": "user", "content": "What is machine learning?"}
)

completion = client.chat.completions.create(
    model=model_deployment,
    messages=conversation_messages
)

assistant_message = completion.choices[0].message.content
conversation_messages.append(
    {"role": "assistant", "content": assistant_message}
)

# Turn 2
conversation_messages.append(
    {"role": "user", "content": "Can you give me an example?"}
)

completion = client.chat.completions.create(
    model=model_deployment,
    messages=conversation_messages # Full history sent
)
```

Advantage: Full control over what context is included

Disadvantage: Manual message management, must resend full history

3.7 Streaming (RESPONSIVENESS)

Responses API Streaming

```
stream = client.responses.create(
    model=model_deployment,
    instructions="You are a helpful assistant.",
    input="Write a short story.",
    previous_response_id=last_response_id,
    stream=True # Enable streaming
)

# Event types to handle
for event in stream:
    if event.type == "response.output_text.delta":
        print(event.delta, end="") # Print partial text
    elif event.type == "response.completed":
        last_response_id = event.response.id # Save response ID
```

Event Types:

- * `response.output_text.delta` - New chunk of text available
- * `response.completed` - Response generation complete

Benefit: User sees output incrementally instead of waiting for full response

ChatCompletions Streaming

```
stream = client.chat.completions.create(
    model=model_deployment,
    messages=conversation_messages,
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="")
```

3.8 Async Patterns

For high-performance applications, use asynchronous clients:

```
from openai import AsyncOpenAI

async_client = AsyncOpenAI(
    base_url=azure_openai_endpoint,
    api_key=token_provider
)

# Use async methods
response = await async_client.responses.create(
    model=model_deployment,
```

```
        instructions="You are a helpful assistant.",
        input="Explain quantum computing.",
        previous_response_id=last_response_id
    )

print(response.output_text)

# Async streaming
stream = await async_client.responses.create(
    model=model_deployment,
    input="Write a haiku about coding.",
    stream=True
)

async for event in stream:
    if event.type == "response.output_text.delta":
        print(event.delta, end="")
```

Benefit: Non-blocking execution, better scalability

4. Azure Services / Tools

Service / Tool	Role	Exam Relevance
Microsoft Foundry portal	Explore projects, models, endpoints and playground	High
Model playground	Test prompts, parameters and generate code samples	High
Microsoft Foundry project	Organizes deployed models and project assets	High
Project endpoint	Access Foundry-native capabilities	High
Azure OpenAI endpoint	OpenAI-compatible model inference endpoint	High
Microsoft Foundry SDK	Access project-level features	High
OpenAI SDK	Build chat inference using Responses or ChatCompletions APIs	High

Service / Tool	Role	Exam Relevance
Azure Identity / DefaultAzureCredential	Enables Entra ID authentication	High
Azure CLI	Supports local authentication through <code>az login</code>	Medium
VS Code	Development environment for the chat app	Medium
GitHub repo/lab files	Provides sample application files	Low–Medium
Azure Key Vault	Secure storage for API keys if key-based auth is used	Medium–High

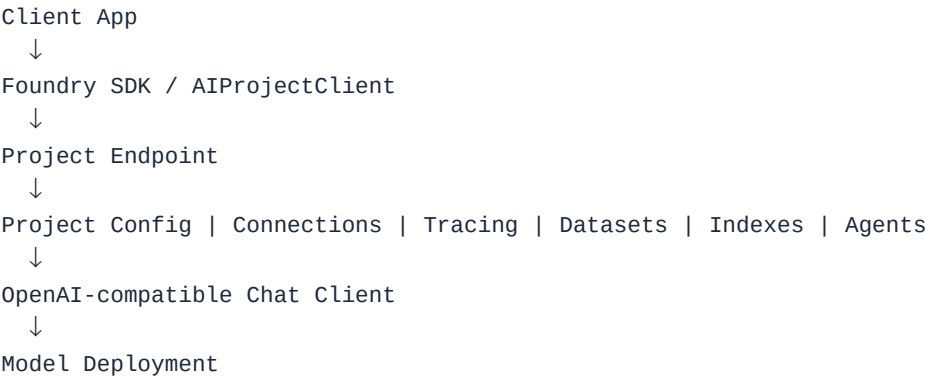
5. Decision Rules

Scenario	Best Choice	Why
Need Foundry agents, tracing, evaluations, datasets or connections	Foundry SDK + Project endpoint	Gives access to project-level Foundry capabilities
Need simple OpenAI-compatible model inference	OpenAI SDK + Azure OpenAI endpoint	Maximum OpenAI API compatibility
Building a new chat app	Responses API	Recommended for most new development
Maintaining existing OpenAI-style chat code	ChatCompletions API	Well-established and widely supported
Need conversation state without manually storing full messages	Responses API with <code>previous_response_id</code>	Simplifies multi-turn context
Need full control over included conversation history	Manual message history	Useful for pruning, persistence and custom context control
Need responsive UI for long answers	Streaming	Displays partial output as it arrives
Need non-blocking or concurrent requests	AsyncOpenAI	Improves responsiveness/scalability
Production authentication	Microsoft Entra ID	Identity-based, more secure than static keys

Scenario	Best Choice	Why
Must use API key	Store in Key Vault / environment variables	Avoid hard-coding secrets
Need no-code prompt testing	Model playground	Validate behavior before coding
Need generated starter code	Playground Code button	Produces endpoint/model/settings-based code samples

6. Endpoint and SDK Architecture

Option A — Foundry-native project features



Option B — OpenAI-compatible inference



7. Exam Traps

Trap	Correct Understanding
Project endpoint and Azure OpenAI endpoint are interchangeable	They are different. Project endpoint is for Foundry project features; Azure OpenAI endpoint is best for broad OpenAI API compatibility

Trap	Correct Understanding
OpenAI SDK gives access to all Foundry project features	No. Use Foundry SDK for agents, evaluations, tracing, datasets, indexes and connections
Foundry SDK is only for model inference	No. Its value is project-level Foundry operations
Responses API and ChatCompletions API are the same	No. Responses API is newer and recommended for most new projects; ChatCompletions is older and message-array based
ChatCompletions automatically remembers conversation context	No. You must manually track and resend message history
Responses API makes token usage free/cheap because it stores previous responses	No. Active context still contributes to token usage
System message and instructions are unrelated	instructions in Responses API serves the same behavioral role as a system message
Streaming improves model intelligence	No. Streaming improves perceived responsiveness by displaying partial output
Async improves answer quality	No. Async improves non-blocking execution and concurrency
API keys are preferred for production	No. Entra ID is preferred where possible
It is safe to put API keys in code	No. Use Key Vault or secure environment configuration
Use ChatCompletions for all new apps	Responses API is recommended for most new Foundry development
The model parameter is the base model name	In deployed Azure/Foundry scenarios, it is commonly the deployment name
Endpoint alone is enough to call the model	You also need authentication and the deployment/model name
DefaultAzureCredential only works locally	No. It works locally (via az login), in CI/CD (via env vars), and in Azure (via Managed Identity)
DefaultAzureCredential tries interactive browser login first	No. It tries environment variables first, then Managed Identity, then az login, THEN interactive browser
All credential types work the same way	No. Each has different use cases: AzureKeyCredential for testing, EnvironmentCredential for CI/CD, ManagedIdentityCredential for Azure resources
You need to choose only ONE credential type	No. DefaultAzureCredential tries multiple methods automatically; this is why it's recommended
Managed Identity requires explicit authentication code	No. It's automatic for resources running in Azure (VMs, App Service, Functions)
InteractiveBrowserCredential is always safe for user-facing apps	It's safe for user login scenarios but creates browser dependency; consider DeviceCodeCredential for IoT
Entra ID scope URL is optional	No. Foundry requires scope https://ai.azure.com/.default when using credential objects

8. Hands-on Checklist

- * ☐ Open the Model playground in Microsoft Foundry
 - * ☐ Test prompts against a deployed model
 - * ☐ Adjust model settings such as temperature and max tokens
 - * ☐ Add system messages/instructions to control model behavior
 - * ☐ Generate code samples from the playground
 - * ☐ Explain the difference between the project endpoint and Azure OpenAI endpoint
 - * ☐ Choose between Foundry SDK and OpenAI SDK
 - * ☐ Explain the 7 Azure credential types
 - * ☐ Explain the DefaultAzureCredential fallback chain
 - * ☐ Determine which credential type to use in different scenarios
 - * ☐ Use `az login` for local AzureCliCredential authentication
 - * ☐ Set environment variables for EnvironmentCredential (AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID)
 - * ☐ Use DefaultAzureCredential in code
 - * ☐ Configure an app with the Azure OpenAI endpoint using DefaultAzureCredential
 - * ☐ Use the OpenAI SDK to call a deployed Foundry model
 - * ☐ Use the ChatCompletions API
 - * ☐ Use the Responses API
 - * ☐ Add conversation tracking using `previous_response_id`
 - * ☐ Explain why context tracking affects token usage
 - * ☐ Implement streaming output
 - * ☐ Use an async OpenAI client
 - * ☐ Clean up Azure lab resources after completion
-

9. Quick Revision Questions

1. What is the purpose of the Microsoft Foundry Model playground?
 2. What can the playground Code button generate?
 3. What are the two main endpoints exposed by a Foundry project?
 4. Which endpoint provides the broadest support for OpenAI APIs with Foundry Models?
 5. When should you use the Foundry SDK?
 6. When should you use the OpenAI SDK?
 7. Which Python package is used for the Microsoft Foundry SDK?
 8. Why is Microsoft Entra ID recommended for production authentication?
 9. What should you do if you must use API keys?
 10. What method is used to generate responses with the Responses API?
 11. What method is used with the ChatCompletions API?
 12. What is the role of the `instructions` parameter in the Responses API?
 13. What is the equivalent of `instructions` in ChatCompletions?
 14. How does the Responses API maintain multi-turn context?
 15. How does ChatCompletions maintain multi-turn context?
-

16. Why can conversation history increase token usage?
 17. What does `stream=True` do?
 18. Why would you use asynchronous API calls?
 19. What three pieces of information does a client app need to call a deployed model?
 20. Why might ChatCompletions still be useful even though Responses API is recommended?
 21. What is the difference between responsiveness and model quality?
 22. In deployed Azure OpenAI/Foundry scenarios, why is the deployment name important?
 23. What is the typical playground-to-code development workflow?
 24. What are the 7 main Azure credential types?
 25. What is DefaultAzureCredential and why is it recommended?
 26. What is the fallback chain for DefaultAzureCredential?
 27. Which credential type should you use for CI/CD pipelines?
 28. Which credential type is automatic for VMs and App Service running in Azure?
 29. When should you use AzureKeyCredential?
 30. What is the difference between InteractiveBrowserCredential and DeviceCodeCredential?
 31. Which credential type uses `az login`?
 32. What environment variables does EnvironmentCredential look for?
 33. What is the Entra ID scope URL for Foundry?
 34. Why is Managed Identity preferred over API keys in Azure?
 35. What happens if you hard-code API keys in your application?
-
-

Final Exam Memory

Remember the decision chain:

Endpoint → SDK → Authentication → API → Context Strategy → Response Pattern

Use this exam shortcut:

Foundry SDK = project features

OpenAI SDK = model inference compatibility

Responses API = new/default for Foundry chat

ChatCompletions API = legacy/widespread compatibility

Entra ID = production authentication

API key = use carefully, store securely

And remember the architecture pattern:

Chat App → SDK Client → Endpoint → Model Deployment → Response

↓

Auth + Context + Streaming/Async

Key Enhancements in This Version

- OK Added endpoint format strings and detailed comparison
- OK Added SDK packages and when to use each
- OK Added all 7 Azure credential types with class names (AzureKeyCredential, DefaultAzureCredential, AzureCliCredential, EnvironmentCredential, ManagedIdentityCredential, InteractiveBrowserCredential, DeviceCodeCredential)
- OK Added DefaultAzureCredential fallback chain in order
- OK Added use cases for each credential type
- OK Added authentication scope URL for Entra ID (<https://ai.azure.com/.default>)
- OK Added production authentication rules and best practices
- OK Added Response object properties (output_text, id, status, usage)
- OK Added Responses API event types (response.output_text.delta, response.completed)
- OK Added code examples for conversation tracking patterns
- OK Added streaming event type specifications
- OK Added async client patterns with await syntax
- OK Added endpoint architecture diagrams
- OK Added authentication-specific exam traps and revision questions
- OK Clarified critical endpoint distinction